

NotebookLM Source: Troubleshooting e Guia de Erros Comuns na Prova

Este guia lista os erros mais recorrentes durante o desenvolvimento e integração da arquitetura, suas causas e como corrigi-los rapidamente.

1. Erro de Mapeamento Duplicado (SQLAlchemy)

O Erro:

```
sqlalchemy.exc.ArgumentError: Class '<class 'domain.Batch'>' already has a primary mapper defined.
```

- **Causa:** A função `start_mappers()` em `database.py` foi chamada mais de uma vez. Isso é comum quando a suíte de testes do `pytest` roda vários testes em sequência e cada teste tenta inicializar a aplicação.

- **Solução:** Adicione um bloqueio com verificação de mapeamentos ativos em `database.py`:

```
def start_mappers():
    # Impede re-mapeamento se o registry já possuir mapeadores registrados
    if mapper_registry.mappers:
        return

    # Código de mapeamento aqui...
```

2. Erro de Instância Desacoplada (DetachedInstanceError)

O Erro:

```
sqlalchemy.orm.exc.DetachedInstanceError: Instance <Batch at 0x...> is not bound to a Session; attribute refresh operation cannot proceed
```

- **Causa:** Você tentou acessar uma propriedade de relacionamento de um objeto de domínio (ex: `batch.allocated_quantity` ou `batch._allocations`) **fora** do bloco `with uow:` (depois que a sessão do banco foi fechada e a transação finalizada).

- **Solução:**

1. Acesse ou extraia as informações necessárias de que você precisa **dentro** do bloco `with uow:`.

2. Configure o mapeador em `database.py` para carregar o relacionamento imediatamente (Eager Loading) usando `lazy="joined"` ou `lazy="subquery"`:

```
"_allocations": relationship(
    lines_mapper,
    secondary=allocations_table,
    collection_class=set,
    lazy="joined" # Carrega os dados na mesma consulta inicial
)
```

3. Importação Circular (Circular Imports)

O Erro:

```
ImportError: cannot import name '...' from partially initialized module '...' (most likely due to a circular import)
```

- **Causa:** O arquivo `domain.py` importa algo de `messages.py`, que por sua vez importa de `handlers.py`, que importa de `domain.py`.
- **Solução (Regra de Ouro da Arquitetura):**
- `domain.py` deve ser **totalmente puro**. Ele **nunca** importa nada de `messages.py`, `handlers.py`, `repository.py` ou `database.py`.
- Se o domínio precisar lançar exceções específicas do negócio (ex: **OutOfStock**), declare-as dentro do próprio `domain.py`.
- Os comandos e eventos em `messages.py` devem conter apenas tipos primitivos (ex: `sku: str`, `qty: int`). Evite importar classes de domínio dentro de `messages.py`.

4. Eventos de Domínio não são disparados pelo Message Bus

O Erro:

Você executa uma ação que deveria disparar um evento (ex: enviar e-mail de falta de estoque), mas o manipulador de eventos (event handler) nunca roda.

- **Causa:** O UOW realizou o commit das alterações no banco, mas a lista de eventos acumulados na raiz de agregado (`domain.Batch.events`) nunca foi lida e enviada para o Message Bus.
- **Solução:** O UOW e os Repositórios precisam monitorar os objetos "vistos" (*seen*) durante a transação. No seu `unit_of_work.py`:

```
class SQLAlchemyUnitOfWork(AbstractUnitOfWork):
    # ...
    def commit(self):
        self._commit()
        self.publish_events() # Dispara os eventos acumulados após o commit

    def publish_events(self):
        # Percorre todas as entidades que o repositório registrou como alteradas
        for entity in self.batches.seen:
            while entity.events:
                event = entity.events.pop(0)
                self.message_bus.handle(event) # Envia ao bus
```

No Repositório (`repository.py`), garanta que toda entidade carregada ou criada seja adicionada a um set de monitoramento:

```
class SQLAlchemyRepository(AbstractRepository):
    def __init__(self, session):
        self.session = session
        self.seen = set() # Monitora os objetos

    def add(self, batch: Batch):
        self.session.add(batch)
        self.seen.add(batch)

    def get(self, reference: str) -> Batch:
        batch = self.session.query(Batch).filter_by(reference=reference).first()
```

```
if batch:
    self.seen.add(batch)
return batch
```

5. Falha nas Restrições de Integridade (Unique Constraint Failed)

O Erro:

```
sqlite3.IntegrityError: UNIQUE constraint failed: batches.reference
```

- **Causa:** A aplicação tentou criar um lote com uma referência que já existe no banco de dados.
- **Solução:** Adicione uma validação no Handler (**handlers.py**) antes de chamar a persistência:

```
def add_batch(cmd: messages.CreateBatch, uow: AbstractUnitOfWork):
    with uow:
        # Verifica se já existe antes de inserir
        existing = uow.batches.get(cmd.ref)
        if existing is not None:
            raise ValueError(f"O lote com referência {cmd.ref} já existe.")

        batch = domain.Batch(ref=cmd.ref, sku=cmd.sku, qty=cmd.qty, eta=cmd.eta)
        uow.batches.add(batch)
        uow.commit()
```

6. Erro de Banco Bloqueado (Database is Locked)

O Erro:

```
sqlite3.OperationalError: database is locked
```

- **Causa:** Múltiplas conexões concorrentes tentando escrever no arquivo SQLite simultaneamente. Isso ocorre se você esquecer de fechar uma sessão (esquecer de sair de um bloco **with uow:** ou criar sessões fora do UOW).
- **Solução:** Garanta que todas as operações com o banco aconteçam exclusivamente dentro do gerenciador de contexto do UOW. Nunca crie instâncias de **Session** diretamente nas rotas da API se puder delegar ao UOW.